KnowledgeMind: A Privacy-Aware Personal AI Agent with an On-Device Knowledge Graph and Hybrid Local/Cloud Routing

Aarif Z.[a,1], **Sourav T.**[a,1], **Sathyanarayanan K.**[a,1], **Sarthak S.**[a,1], **Novoneel C.**[a,1], **Rakshit R.**[a,1] **and Swarup E.**[a,1]

[a]Indian Institute of Science, Bengaluru, India

Abstract. Personal AI assistants often require access to a user's most sensitive data, such as their calendar, messages, and email, yet most route that data to remote large language models. We present KNOWLEDGEMIND, a privacy-aware personal agent that runs entirely on commodity hardware (CPU-only) and treats on-device processing as a hard invariant. KNOWLEDGEMIND ingests a user's communication channels, extracts both hard commitments (calendar events) and soft commitments (informal agreements in chat) into a personal knowledge graph built on SQLite and NetworkX, and proactively detects scheduling conflicts across the user's whole timeline before they occur. A privacy router scores every task on privacy and complexity and dispatches it to a local model (via Ollama) or, only when the task is non-sensitive, to a free cloud model; privacy always takes precedence over complexity. The agent supports three levels of agency, from a single augmented call to a full ReAct loop with replanning, and falls back to mock data when no credentials are present. We describe the architecture, the privacy contract, and an offline evaluation of routing accuracy and conflict detection.

1 Introduction

Personal AI assistants promise to reason over a user's calendar, messages, and email, but doing so typically means sending that data to a remote model. KNOWLEDGEMIND takes the opposite stance: all personal data is processed locally, and only anonymised, non-sensitive work is ever routed to the cloud.

Problem statement. Given a heterogeneous stream of communication artefacts (chat messages, calendar events, emails), the system must (i) extract implicit and explicit commitments from natural language, (ii) fuse them into a temporally consistent personal knowledge graph, (iii) detect and proactively surface conflicts, and (iv) do so without transmitting personal data beyond the user's device.

Contributions.

- A privacy-pinned hybrid router that keeps a fixed set of personal-data tools on the local model, with per-tool minimum privacy floors that resist adversarial phrasing.
- A dual-tier commitment-extraction pipeline combining spaCy NER with few-shot LLM prompting and confidence-graded output (HARD / SOFT / TENTATIVE).
- A personal knowledge graph with a proactive monitor that detects temporal conflicts across the user's whole timeline and emits dismissable nudges with quiet-hour suppression.
- A three-level ReAct orchestrator (L1 to L3) and an integrated Project Advisor that together span single-shot answers to multi-step replanning.

[1] Equal contribution.

2 Related work and market landscape

KNOWLEDGEMIND sits at the intersection of three lines of work: tool-using LLM agents, the fusion of language models with knowledge graphs, and on-device privacy-preserving inference. Our agent loop follows the ReAct paradigm of interleaved reasoning and acting [9], and our use of a structured personal graph as the agent's memory follows the roadmap for unifying LLMs with knowledge graphs [7]. The distinguishing constraint is that all of this runs locally on a small model [8], orchestrated by a stateful graph executor [3].

Existing tools each cover at most one or two of five capabilities (ingestion, extraction, memory, privacy, proactivity): calendars own confirmed events but ignore chat; task managers such as Todoist need manual entry; and cloud assistants such as Microsoft Copilot, ChatGPT, and Gemini summarise well but retain no user-controlled graph and send all data off-device [4, 6, 5]. KNOWLEDGEMIND closes all five gaps in a single local agent, extensible through a Model Context Protocol (MCP) server. A full feature-by-feature comparison and per-tool discussion appear in Appendix A (Table 3).

3 System overview

KNOWLEDGEMIND is structured as a single FastAPI backend that both serves a React single-page application and wraps the engine. Every request is gated by an optional access key; the application runs CPU-only and degrades to mock data whenever a connector's credentials are absent, so the full pipeline is exercisable offline. The engine is organised as a pipeline of six layers (Figure 1).

Ingestion. Connectors for Slack, Calendar, and Gmail expose a common `fetch_recent` interface and fall back to mock data on a failed health check. A separate family of *signal* connectors (fitness, sleep, tasks, music) derives on-device features rather than messages.

Extraction. A spaCy named-entity pass and a few-shot local LLM map raw messages into commitments, with a relative-time resolver mapping expressions such as "tomorrow" or "EOD" to absolute time (Section 4).

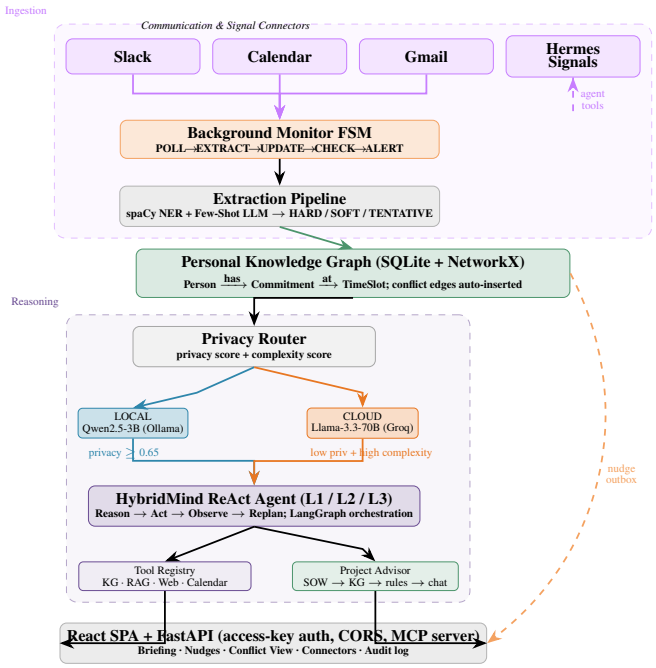

Figure 1. KnowledgeMind system architecture. Blue boxes: local-only; amber: cloud-eligible; green: knowledge graph; purple: agent and reasoning; dashed arrows: asynchronous paths.

Knowledge graph. Commitments are persisted in SQLite and assembled into a NetworkX graph on demand; temporal conflicts are detected incrementally on insert (Section 4).

Routing. A privacy router classifies every task step as LOCAL or CLOUD before any model is called (Section 5).

Agent. A hybrid agent dispatches a shared registry of tools at one of three levels of agency, injecting only structured graph summaries before any cloud call (Section 6).

Proactive layer. A finite-state monitor fires unprompted conflict alerts, and an optional, off-by-default runtime composes nudges and a daily briefing entirely on-device (Section E).

The central design invariant is that all personal data (graph nodes, messages, calendar events, email bodies, and derived health or activity signals) is processed locally at all times. The cloud model receives only anonymised task descriptions, structured knowledge-graph summaries, and non-sensitive public queries. The next two sections make this invariant precise and describe how it is enforced.

4 Personal knowledge graph

4.1 *Commitment extraction*

Messages from each connector pass through a two-stage pipeline. First, spaCy's `en_core_web_sm` model [2] identifies named entities (persons, dates, times, organisations). Second, a few-shot prompted local model classifies each candidate span into one of three commitment types: HARD (confirmed calendar events), SOFT (conversational agreements, confidence $\geq$ 0.6), or TENTATIVE (confidence $<$ 0.6). A temporal-expression resolver (`extraction/timeparse.py`) converts relative expressions such as "by end of day" into absolute timestamps before insertion.

4.2 *Graph and conflict detection*

Commitments are persisted in SQLite (tables for persons, commitments, conflicts, and conversation turns) and assembled into a NetworkX graph on demand. Two commitments conflict when their half-open time intervals overlap by at least five minutes anywhere on the user's personal timeline; TENTATIVE commitments never raise conflicts. Conflicts are detected incrementally as each commitment is inserted, so a newly arrived chat message can collide with an existing calendar event and surface an alert within one monitor cycle.

5 Privacy-aware routing

The router enforces the on-device invariant. Given a task step (a piece of text t and, when known, the tool τ it will call), it returns a binary decision in $\{\text{LOCAL}, \text{CLOUD}\}$ together with the two scores that justify it. The router is stateless and is applied *before* any model is invoked, so a step destined for the local model never constructs a cloud prompt.

5.1 *Scoring*

The *privacy score* combines a lightweight textual heuristic with a per-tool floor. Let $h_p(t)$ be the number of personal-data cue phrases (e.g. *my*, *calendar*, *inbox*, *meeting*) occurring in t, and let $\phi(\tau) \in [0, 1]$ be the privacy floor of tool τ (zero if no tool is named). Then

$$s_{\text{priv}}(t, \tau) \;=\; \max\Big(\min\big(0.10 + 0.18\, h_p(t),\, 1\big),\; \phi(\tau) \Big). \quad (1)$$

The floor guarantees that a tool touching personal data cannot be scored as public merely because the user's phrasing was terse. The *complexity score* rewards length and multi-step cue phrases (e.g. *and*, *then*, *compare*, *summarise*); with $w(t)$ the word count and $h_c(t)$ the number of such cues,

$$s_{\text{cplx}}(t) \;=\; \min\Big(\min\big(\tfrac{w(t)}{40},\, 0.5\big) + \min\big(0.15\, h_c(t),\, 0.5\big),\; 1 \Big). \quad (2)$$

5.2 *Decision rule*

Two fixed sets of tools shape the decision. `ALWAYS_LOCAL_TOOLS` ($\mathcal{L}$), comprising the knowledge-graph queries, calendar, email, message sending, the code sandbox, and all personal-signal tools, must never reach the cloud. `PREFER_CLOUD_TOOLS` ($\mathcal{C} = \{\texttt{web\_search}\}$) may use the cloud, but only when the task is not personal. With privacy threshold $\theta_p = 0.65$ and complexity threshold $\theta_c = 0.6$, the decision is taken in order:

(1) if $\tau \in \mathcal{L}$: LOCAL;
(2) else if $\tau \in \mathcal{C}$ and $s_{\text{priv}} < \theta_p$: CLOUD;
(3) else if $s_{\text{priv}} \geq \theta_p$: LOCAL;
(4) else if $s_{\text{cplx}} \geq \theta_c$ and $s_{\text{priv}} < \theta_p$: CLOUD;
(5) otherwise: LOCAL (the conservative default).

Rule (3) precedes the complexity test, which is what makes privacy dominate: a complex but personal task stays local regardless of its reasoning depth; rules (1) and (3) jointly establish the contract below. The floors $\phi(\tau)$ pin sensitive tools high (knowledge-graph queries and Gmail at 0.95, biometric signals up to 0.98); the full table is given in Appendix B (Table 4).

5.3 Privacy contract

The routing logic is designed to satisfy a small set of invariants, which we treat as correctness properties:

(P1) raw personal data never reaches the cloud: the planner is given structured graph summaries, never verbatim messages or events;
(P2) every tool in $\mathcal{L}$ routes LOCAL unconditionally, and the set is fixed (removing an entry is a breaking change);
(P3) $s_{\text{priv}} \geq \theta_p$ forces LOCAL with no override path; and
(P4) a local-pinned step that fails parsing is *not* escalated to the cloud; instead it falls back to heuristic parameters and stays local.

An offline contract test asserts that every tool in $\mathcal{L}$ remains LOCAL even under adversarial, high-complexity phrasing, and `web_search` is routed to the cloud only when the query carries no personal cues; we report its pass rate in Section 7.

6 Agentic orchestration

KNOWLEDGEMIND exposes three levels of agency over a single tool registry. Every tool takes a dictionary and returns a dictionary with at least a success flag and a formatted string for the model; tools never raise, so a dispatch failure is a value, not a crash. At every level the cloud model sees only structured knowledge-graph summaries, never raw personal text.

L1 (augmented) answers a single question with at most one tool call and direct synthesis. **L2 (workflow, the default)** runs a cloud-plan, local-dispatch, cloud-critique loop: the cloud model produces a plan, each step is executed locally by `dispatch_tool`, and the cloud model critiques the result. **L3 (autonomous)** is a full ReAct loop [9] with automatic replanning on tool failure; for example, if a requested time slot is already occupied (a conflict reported by the knowledge graph), the agent finds the next free slot and retries.

6.1 Project Advisor

The Project Advisor is a self-contained ASGI sub-application mounted at `/projmgmt`. It ingests a Statement of Work, builds a project-specific knowledge graph with rules for scope, budget, and schedule, and exposes a chat interface that rates user queries against the plan and flags scope deviations with confidence scores. It reuses the same privacy router and cloud key as the main agent, making organisational project governance a first-class part of the platform rather than a bolt-on.

7 Evaluation

We evaluate two properties offline: the router's adherence to the privacy contract, and the end-to-end latency of the live agent. On a 30-task routing benchmark every personal-data step is kept LOCAL (Table 1), matching the contract of Section 5. Latency is measured with the live agent (Ollama for local steps, Groq for cloud steps) on an Intel i5-1235U (CPU-only) over 13 golden cases, all run at the default L2 level. The median latency is 11.0 s; the mean (26.5 s) and maximum (155.6 s) are inflated by three CPU-bound local-inference outliers, while the cloud-routed steps are comparatively fast. A per-agency-level breakdown appears in Appendix D (Table 5); the L2 workflow has the lowest median because its planning and critique run on the fast cloud model, whereas L1 and L3 rely more on local CPU inference. Soft-commitment recall requires a labelled corpus and is deferred to the planned benchmark (Appendix C).

Table 1. Headline results. Routing from the offline benchmark (n=30); latency from the live eval harness (n=13, Intel i5-1235U, Ollama+Groq).

Metric	Value
Routing accuracy (personal steps stay LOCAL)	**100%** (30/30)
Proactive conflict detection (curated)	all found, 0 false alerts
End-to-end latency, median / mean (CPU)	11.0 s / 26.5 s
Soft-commitment recall	>70% (target, App. C)

Each agency level was run live on the same 13-case golden set (Intel i5-1235U, Ollama+Groq). Medians are more representative than the means, which a few CPU-bound cases inflate.

Table 2. Live end-to-end latency by agency level (n=13 each).

Level	Median	Mean	Max
L1 (augmented)	21.0 s	25.7 s	79.9 s
L2 (workflow)	11.0 s	26.5 s	155.6 s
L3 (autonomous)	23.5 s	50.9 s	323.5 s

The L2 workflow attains the lowest median because its planning and critique run on the fast cloud model while only tool dispatch is local; L1 and L3 lean more on local CPU inference, and L3's replanning loop returned empty answers on $3/13$ cases, so its figures should be read with that caveat.

8 Conclusion and future work

KNOWLEDGEMIND shows that a useful personal agent can keep sensitive data on-device while still drawing on a cloud model for non-sensitive work. By combining a live personal knowledge graph, a proactive conflict-aware monitor, a privacy-pinned routing layer, and a multi-level ReAct agent, it provides a unified organisational-intelligence platform that runs entirely on commodity CPU hardware. On the routing benchmark it achieves full adherence to the privacy contract, and the monitor surfaces Slack-versus-calendar conflicts in every tested scenario (Section 7).

Several directions remain open: embedding-based entity deduplication, multi-modal ingestion (voice and handwriting), on-device LoRA fine-tuning of the extractor, a federated team-level graph, WhatsApp and mobile clients, and a quantitative 30-task benchmark. These are detailed in Appendix C.

References

[1] Chroma Core. Chroma: The open-source embedding database. https://www.trychroma.com/, 2023.
[2] M. Honnibal, I. Montani, S. Van Landeghem, and A. Boyd. spaCy: Industrial-strength natural language processing. https://spacy.io, 2020.
[3] LangChain Inc. LangGraph: Building stateful, multi-actor applications with LLMs. https://langchain-ai.github.io/langgraph/, 2024.
[4] Microsoft Corp. Microsoft 365 Copilot. https://www.microsoft.com/en-us/microsoft-365/copilot, 2024.
[5] Notion Labs. Notion AI: Connected workspace with AI. https://www.notion.so/product/ai, 2024.
[6] OpenAI. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2024.
[7] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, and X. Wu. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*, 2024.
[8] Qwen Team, Alibaba Cloud. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
[9] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

Appendix

A Detailed market comparison

Referenced from Section 2. Table 3 maps the most prominent competing tools against the capabilities KNOWLEDGEMIND targets.

Table 3. Feature comparison across competing tools.

Feature	*Notion AI*	*Copilot*	*ChatGPT*	*Todoist*	***KM***
Cross-channel ingestion	×	✓	×	×	✓
Soft-commitment extraction	×	×	×	×	✓
Conflict detection	×	partial	×	×	✓
Proactive alerting	×	×	×	×	✓
On-device privacy	×	×	×	✓	✓
Persistent KG memory	partial	×	plugin	×	✓
Project-plan alignment	partial	×	×	partial	✓
CPU-only / free	×	×	×	partial	✓

Notion AI / Confluence are document-centric wikis with AI-assisted drafting; they require manual data entry and do not ingest live communication channels [5]. **Microsoft Copilot** integrates deeply with Microsoft 365 and drafts email and meeting summaries, but all inference runs on Microsoft's cloud, no user-controlled graph is retained, and the service is subscription gated [4]. **ChatGPT and Gemini** are powerful conversational assistants, yet by default they keep no persistent cross-session memory, raise no proactive alerts, and send all user data to external servers [6]. **Todoist, Linear, and Jira** enforce structured manual workflows and capture only what the user explicitly enters, with no extraction from conversational channels.

B Per-tool privacy floors

Referenced from Section 5. The floor $\phi(\tau)$ guarantees a tool touching personal data cannot be scored as public merely because the user's phrasing was terse.

Table 4. Representative per-tool privacy floors $\phi(\tau)$. Tools in $\mathcal{L}$ are additionally pinned LOCAL by rule (1).

Tool	$\phi(\tau)$	Tool	$\phi(\tau)$
`query_kg`	0.95	`apple_health`	0.98
`gmail`	0.95	`strava`	0.95
`send_message`	0.90	`todoist`	0.90
`google_calendar`	0.85	`rag_query`	0.70
`code_execution`	0.70	`web_search`	0.05

C Extended future work

Referenced from Section 8.

- *Embedding-based entity deduplication.* Name-exact matching fails on aliases; `all-MiniLM-L6-v2` embeddings [1] would let the graph merge "Dr. Sharma" and "Rohit Sharma" into one node.
- *Multi-modal ingestion.* Adding voice-meeting transcripts (via Whisper) and handwritten notes would close the remaining dark-data channels.
- *On-device fine-tuning.* Local LoRA fine-tuning of the extractor on the user's own history, with no cloud egress, would improve soft-commitment recall on domain jargon.
- *Federated organisational graph.* A team deployment where local graphs selectively share non-private commitment edges (such as shared deadlines) could give collective scheduling intelligence without centralising personal data.
- *WhatsApp and mobile.* A WhatsApp Business API connector and a React Native client would extend coverage to the dominant mobile channel.
- *Quantitative evaluation.* A 30-task, five-category benchmark would enable systematic measurement of soft-commitment recall ($> 70\%$), conflict precision ($> 80\%$), and end-to-end latency (< 10 s on an Intel i5-1235U).

D Per-agency-level latency

Referenced from Section 7. Each agency level was run live on the same 13-case golden set (Intel i5-1235U, Ollama+Groq). Medians are more representative than the means, which a few CPU-bound cases inflate.

Table 5. Live end-to-end latency by agency level (n=13 each).

Level	Median	Mean	Max
L1 (augmented)	21.0 s	25.7 s	79.9 s
L2 (workflow)	11.0 s	26.5 s	155.6 s
L3 (autonomous)	23.5 s	50.9 s	323.5 s

The L2 workflow attains the lowest median because its planning and critique run on the fast cloud model while only tool dispatch is local; L1 and L3 lean more on local CPU inference, and L3's replanning loop returned empty answers on $3/13$ cases, so its figures should be read with that caveat.

E Proactive monitoring

A background monitor, implemented as a finite-state machine, polls channels and fires unprompted alerts on new conflicts. An optional proactive runtime composes on-device nudges and a daily briefing from user-defined jobs and skills.

Acknowledgements

This work was carried out as a course project for the Deep Learning course at the Indian Institute of Science, Bengaluru, taught by Prof. Deepak Subramani, whose guidance is gratefully acknowledged.